CHANGELOG

# HORIZON GRID

Changelog

**Version:** 0.3.0
**Documentation prepared:** 2026-08-24

PREPARED FOR EVALUATION

# Table of Contents

HORIZON GRID Changelog 3
Linux Release 14

# HORIZON GRID Changelog

All notable changes for the HORIZON GRID release are listed below, grouped by area. Every entry below reflects a real, tested change — not a planned or aspirational one.

## v0.3.0 — Pentest Suite: scope-enforced assessments and gated real-exploit validation

A new, standalone assessment lifecycle — DISCOVER → ENUMERATE → ASSESS → CORRELATE → PRIORITIZE → REPORT — built as a second orchestration layer over the same tool adapters (Nmap, DNS, TLS, HTTP headers, hash analysis) the existing per-investigation Security Assessment Toolkit already provides, targeting its own standalone assessments/targets/findings instead of a single `IOCLookup`. The Security Assessment Toolkit itself, and its UI, are completely untouched.

**Assessments, scope, and control.** An assessment declares its own scope up front (CIDR ranges and/or domains) — an empty scope authorizes nothing, and a target outside the declared scope is rejected outright with no way to expand scope mid-scan. Four preset profiles (Passive/Low Impact/Standard/Comprehensive) plus a Custom tool-list option. A live control panel supports pause/resume/cancel/emergency-stop per assessment, plus a global, admin-only kill switch that halts every running assessment platform-wide. Every terminal-state database write uses the same conditional-update concurrency guard ( `UPDATE ... WHERE status IN (...)` ) the Security Assessment Toolkit's own cancellation-race fix established — reused verbatim, not reinvented, so a pause/cancel racing a completing background scan can never clobber a status another writer already finalized.

**Findings carry a confidence rating separate from severity** — `Confirmed` / `Likely` / `Potential` / `Informational` — because "how bad" and "how sure this platform is that it's real" are genuinely different questions. A CVE match from a version-string scan is `Likely`, never auto-promoted to `Confirmed`; that status is reserved for a non-destructive "Intrusive Validation" re-check (re-runs the same tool, confirms the same finding reappears, no payloads, no new ports) or a real, observed fact from Exploit Validation below.

**AI Security Analyst** narrates real findings — a plain-language explanation per finding, a grounded executive/technical summary across an entire assessment — following the same discipline this platform's core AI layer already applies: never invent a finding, a target, or a number not present in the real data it was given. Live-verified: the summary's own "top priorities" list is filtered against real finding titles, and did in fact drop an AI-invented title that didn't match anything real during testing.

**Exploit Validation (Metasploit) — the one genuinely new, real capability.** Administrators only, and only for a finding that already has a matched CVE. A real, self-hosted Metasploit Framework connection (the actual Rapid7 omnibus package, baked into the backend's own Docker image at build time — a large, deliberate ~1-2 GB addition, not a transitive dependency) is driven through a hand-rolled msgpack-RPC client, itself driving a real `msfconsole` session via the RPC `console.*` methods rather than the lower-level job API, specifically so the stored evidence is the genuine, complete console transcript a human typing into `msfconsole` would see — including Metasploit's own real `check` command response (Vulnerable / Not Vulnerable / Cannot Reliably Check), never a synthesized verdict.

Five independent gates stand between "an assessment exists" and "a real exploit ran," with no shortcut past any of them: a declared scope; a target inside it; a completed scan that produced a CVE-matched finding; an administrator manually searching and picking one specific module for that one finding; and, for real execution specifically (not the non-exploiting `check` mode), an explicit confirmation ticked on that exact request, required fresh every single time. The target host ( `RHOSTS` / `RHOST` ) is always force-overwritten with the finding's own real target regardless of what a caller supplies — verified, including under adversarial review, to hold even against homoglyph or whitespace-padded option-key tricks, because the real value is always inserted last into the options dict and a Python dict's insertion order guarantees it's always the final assignment sent to the console. Every option value is also rejected outright if it contains a newline or carriage return, closing a console-command-injection path a crafted value could otherwise use.

**A dedicated adversarial security review** — an independent pass reading the finished code fresh and trying concretely to break each stated guarantee — ran before release. It confirmed the RHOSTS lock, the mandatory confirmation gate, and the scope/kill-switch re-checks all hold, found one real, confirmed bug (the list of past exploit attempts, which can contain genuine post-exploitation transcripts including dumped credentials or session output, was gated on the broader `pentest:read` permission that Analyst and Viewer roles also hold, instead of the admin-only `pentest:exploit`), and surfaced several worthwhile hardening opportunities. All were fixed before release: the transcript-list route is now `pentest:exploit`-gated; the exploit-running service function now also re-checks the caller's role itself, not only via the API route's own permission dependency; the global kill switch is now re-checked on every console poll tick during a running check/exploit (roughly once per second) rather than only before it starts, so engaging it now aborts in-flight work within about a second instead of only blocking the next action; and a target/assessment consistency check was added as cheap extra insurance. Every fix is covered by a new regression test, including one that specifically reproduces the confidentiality bug and confirms it's closed.

**Two further real bugs found and fixed during live verification against an actual running Metasploit instance** (not caught by the mocked test suite, which never talks to a real `msfrpcd`): Metasploit's own Ruby-side msgpack encoder emits old-spec "raw" strings rather than the modern string formats, which Python's `msgpack` library — even configured correctly — still decodes as raw bytes rather than text; every field this client read (including `auth.login`'s own `"token"` key) was silently bytes instead of a string, permanently reporting Metasploit as unreachable even while it was running correctly. Fixed with a recursive bytes-to-text normalizer, now covered by its own regression test. Separately, the shared `Dialog` UI component (used by several existing panels, not just this feature) had no maximum height or scroll handling at all; a real Metasploit module's full option list is often dozens of fields, long enough to push the dialog's own action buttons below the visible screen with no way to reach them for any user, not just an automated test. Fixed at the component level, benefiting every dialog in the app going forward, not only this one.

**Testing.** The backend test suite grew from 404 to 432 passing tests across this work (39 pre-existing skips and 2 pre-existing pytest-collection artifacts, both unrelated to this release and documented in the Security appendix), zero regressions. Beyond the mocked suite, this was verified live end-to-end against a real, running Metasploit installation: a real module search by CVE (returned real, correctly-ranked, correctly-labeled results for a well-known vulnerability), real module option/metadata retrieval, and a real non-exploiting `check` run against an authorized loopback test target that had no listening service — correctly and honestly reported as "cannot reliably check exploitability" rather than a fabricated result either way.

## v0.2.5 — Security Assessment panel visibility and friction fixes

Two real bugs found and fixed while live-testing the Security Assessment Toolkit (the port scanner UI) against a fresh install. Both frontend-only: no backend logic, database schema, or API contract changed.

**Panel visibility:** the entire Security Assessment panel — including its "Run" trigger form — silently disappeared with no explanation whenever an investigation's overall status ended up `FAILED`, even when every provider had succeeded. It was gated on `status === "completed"` alongside other panels (Verdict Analysis, Evidence, Pivot, Hunting Center, Copilot) that genuinely depend on a successful AI final assessment — but Security Assessment doesn't read `finalAssessment` at all, so a failed final-AI-call (e.g. the configured AI backend being temporarily unreachable) had nothing to do with whether this unrelated active-scan feature should be available. Now renders on either terminal state (`completed` or `failed`); the panels that do depend on the final assessment keep the original gate.

**Confirmation friction:** the target-confirmation field required retyping the exact IOC value with zero feedback on any mismatch (no trim, no case-insensitivity, no error message) — a single stray space silently kept the "Run Security Assessment" button disabled forever, indistinguishable from a real bug. The retype step added friction without adding real protection beyond the existing authorization checkbox, since the backend already scopes every run to the specific investigation's own IOC regardless of what's typed. Replaced the input with a plain read-only "Target: `<value>`" line; the one authorization checkbox is now the sole confirmation gate.

**Testing:** both bugs reproduced live against a real investigation and a real Nmap scan (not just unit tests) — confirmed broken before the fix, confirmed working (panel visible, scan runs and completes with zero typing) after. Full backend regression suite re-run clean after each change: 383 passed, 39 skipped, zero regressions. The Windows installer was verified via an isolated silent install confirming the fix is actually packaged into the shipped frontend source, then fully cleaned up (uninstalled, registry entry removed).

## v0.2.4 — Original visual identity and branding pass

A professional branding + UI/UX pass across the entire frontend, requested because the product name and tagline ("Every Signal. One Operational Picture.") were previously barely visible anywhere in the running application. Presentation-only by design: no backend logic, database schema, authentication, IOC processing, AI provider logic, provider API logic, port scanner logic, or admin permissions were touched, verified by a full backend regression run (383 passed, 39 skipped) and a clean frontend typecheck both before and after every change.

**Visual identity ("Datum Signal"):** a CRT-phosphor teal-cyan primary color, deliberately pulled off the generic-SaaS-blue hue almost every dark-mode dashboard defaults to, against a near-black anodized-steel-panel shell with a warm off-white foreground. Colors are the same set of HSL CSS variables the app already used, so the identity is a drop-in palette change, not an architecture rewrite. A six-state operational status language (OPERATIONAL/DEGRADED/WARNING/CRITICAL/OFFLINE/UNKNOWN) replaces ad-hoc status badges wherever they existed — every state pairs a distinct hue, fill density, border style, AND icon, deliberately never color alone, so removing color entirely (a colorblind viewer, a grayscale printout) still leaves every state distinguishable.

**Logo:** an original, wholly geometric mark — a horizon line with two open "signal" nodes converging on one filled "detection" node — chosen specifically because it's the only concept considered that actually depicts the tagline rather than just looking generically tactical. No insignia, seal, shield, or any existing company/military symbol. Ships as a full lockup (mark + wordmark), a compact lockup (mark + "HG" in a corner-bracket badge — the same bracket motif reused as a UI chrome accent elsewhere), and a favicon.

**Typography:** IBM Plex Sans Condensed for headings/wordmark/section labels, IBM Plex Mono for every raw technical value (hashes, IPs, timestamps, coordinates, counts, scores) with tabular numerals, and the existing body typeface left completely untouched — the highest-risk, lowest-payoff move available (re-tuning every dense data table to a new body face's metrics) was deliberately avoided.

**New page:** `/about` — version, live dependency status (from the real health endpoint, nothing hardcoded), supported platforms, documentation links. Purely additive; no existing route was touched to add it.

**Component-level changes that took effect app-wide from one edit each:** the radius-contrast rule (outer panels keep the existing softer corner radius; every interactive/status element — buttons, inputs, badges, chips — tightens to a sharper radius, so soft panels visibly contain precise controls); dropped card drop-shadows in favor of a flat fill bounded by a hairline border; section/column headers now default to uppercase, tracked, muted "instrument-panel" labels instead of bold sentence-case headings.

**Reports:** every exported PDF now carries a HORIZON GRID header/footer with real page numbering, a generation timestamp, and the Investigation ID; CSV and Markdown exports gained the same platform/Investigation-ID fields.

**Real bug self-discovered during screenshot QA, fixed:** the new `/about` page initially read client-only login state directly in its render body instead of via `useState` + `useEffect`, producing a genuine React hydration-mismatch error whenever a session already existed at first paint. Fixed by deferring the check to a post-mount effect, matching the pattern already used correctly elsewhere in this codebase.

No manual upgrade step is required; re-running the installer/package on an existing install preserves your configuration and data as always.

## v0.2.3 — Mission-critical deployment hardening: 18 real gaps found and fixed

A dedicated reliability and security review for unattended, remote-site deployment (installed once, expected to run correctly for a long time with no developer access afterward). Every item below was confirmed present before the fix (live reproduction or direct code-path tracing) and confirmed resolved after (a real test, a live re-verification, or both) — nothing here was assumed fixed on intention alone. Two gaps were self-discovered during this review, not flagged by the initial structured assessment.

**Reliability and recovery:**

- **Only 2 of 8 Docker Compose services auto-restarted on a crash.** `backend`, both datastores that matter operationally (Postgres, Redis), Neo4j, OpenSearch, and the frontend had no restart policy at all — a crashed or OOM-killed container stayed down until a human ran `docker compose up`. Every service now has `restart: unless-stopped`. Live-verified: a container OOM-killed by its own memory limit now restarts automatically within seconds.
- **A real dependency-aware health endpoint, `GET /health/detailed`, added.** The existing `/health` is a pure liveness check — it returns `ok` the instant the process accepts connections, even with Postgres fully stopped, and is deliberately kept unchanged (CI's bare test job and simple reachability probes depend on it staying dependency-free). `/health/detailed` genuinely pings Postgres and Redis, returning `503` / `"down"` if Postgres is unreachable and `"degraded"` if only Redis is. Live-verified by actually stopping the Postgres container and watching the correct 503 response, then restarting it and watching recovery.
- **Boot-time auto-start added on both platforms.** Previously, nothing restarted the platform after a host reboot — a power-loss-induced restart at a remote, physically inaccessible site left it down indefinitely. Windows: a `SYSTEM`-level Scheduled Task fires once at boot. Linux: `systemctl enable horizon-grid.service` is now actually called by the setup wizard — **a real, previously-undiscovered bug found during this review**: the systemd unit file's own `WantedBy=multi-user.target` declaration does nothing until `systemctl enable` creates the real symlink, and an exhaustive search confirmed that call existed nowhere in the codebase. A fully configured, working Linux install would silently not survive a reboot.
- **A 5-minute health watchdog added on both platforms**, backstopping the case where containers are still "running" but the application inside them is genuinely broken (a hung process, a config error that leaves it unable to reach Postgres even though Postgres itself is healthy). Checks `/health/detailed`; on failure, restarts and logs the outcome. A healthy check produces no log line, so the watchdog log only grows when there's something worth an operator's attention.
- **Fixed a real bug in the health-check port detection** (both platforms): the watchdog and every unattended health caller hardcoded port 8000/3000 regardless of a customized `HOST_PORT_BACKEND` / `HOST_PORT_FRONTEND` (set if the default conflicted with something already running on the machine) — every unattended caller was checking the wrong port forever on a non-default install. Now reads the real configured value. Live-verified with a dummy HTTP server on a non-default port.
- **Redis now persists across a container restart.** Confirmed live before this fix: a value set, then a routine reconfigure-triggered container recreation, silently reset it to empty — resetting every rate limiter and the whole provider-result cache on an otherwise ordinary config change.

**Backup and disaster recovery:**

- **A real restore procedure added on both platforms** — previously only backup scripts existed anywhere, and the one documented manual restore procedure was self-contradictory (it said to stop the platform, then restore into "the running Postgres container" — but stopping the platform stops Postgres too). The new scripts stop only the services that write to the database, leave Postgres running, drop/recreate the target database (required for a plain `pg_dump` with no `--clean` to replay cleanly), restore, and restart what they stopped — with an explicit typed confirmation required unless `--force` / `-Force` is passed. Live-verified end to end: backed up, inserted a marker row, restored, confirmed the marker gone and the platform healthy afterward.
- **Scheduled daily database backups added on both platforms** (previously manual/pre-upgrade-only on Windows, manual-only on Linux). A pre-upgrade backup was also added to the Linux wizard for parity with the existing Windows behavior.

**Security:**

- **SSRF guard extended to the actual AI-call path.** The link-local-address check (blocks cloud instance-metadata services at `169.254.169.254`) previously ran only when an operator clicked "Test Connection" — the real call path every investigation uses was unchecked, including the common case of a wizard-driven install that never touches the runtime-config web panel afterward (a completely separate code path reading the frozen `.env` setting, which had no check at all). Centralized inside the one real client-construction choke point so both paths are covered.
- **Login brute-force rate limiting added** — a fixed-window limit (10 attempts/60s by default) keyed on the attempted email, checked before password verification. Deliberately a rate limit, not a hard account lockout, which would itself be a way to lock out a real administrator.
- **Swagger UI, ReDoc, and the raw OpenAPI schema are now disabled in production** (`ENVIRONMENT=production`, set by `docker-compose.prod.yml`, the override every real installer uses) — previously exposed unconditionally, handing anyone who

could reach the API a full map of every route and request/response shape.

- **A global 10 MB request-body-size limit added**, plus `max_length` constraints on every previously-unbounded free-text field (investigation IOC value, case description/note body) — none had any limit before.

**Provider and AI resilience:**

- **Fixed a dead-retry-code bug**: `BaseProvider.run()` was silently catching the exact exception types the orchestrator's retry loop was meant to see and retry, so `provider_max_retries` had no real effect for any provider that didn't implement its own network-error handling. Live-verified with a fake provider: a transient connection error is now genuinely retried, where before it was not.
- **Fixed a real correctness bug in the Final Assessment panel**: a genuine AI generation failure and a correct "no evidence to assess" decision rendered the exact identical badge — the backend already tracked which case applied and sent it in every response, but the frontend's type never declared the field, so nothing read it. Confirmed against a real failed-AI-generation record from this session's own testing.
- Added a concurrency cap on security-assessment (port scan) execution and CIDR-size-proportional nmap timeout scaling (a full /28 previously got the identical wall-clock budget as a single host).

**Installation:**

- **Both setup wizards now gate configuration save on a real Test Connection result**, blocking "Next" only when the exact current value was just live-tested and failed (never on "untested" — a fresh install has no backend running yet to test against, a hard structural constraint, not an oversight). The install flow's own Summary/Install page additionally runs an automatic post-health-check AI connectivity test once the backend exists, reporting the honest result rather than silently assuming a saved configuration works.
- **Linux's prerequisite check is now a real blocking gate** in the setup wizard, not an ignored informational check (the previous `.deb` postinst discarded its exit code entirely). **A real, previously-undiscovered bug found and fixed during this review**: the check script's own JSON output had a permanently empty `checks` array in every run — its helper function interpolated bash's lowercase `true` / `false` directly into generated Python source as bare identifiers, which Python doesn't recognize as booleans, so every call silently failed via its own error-swallowing fallback. The human-readable console output and the script's overall exit code were computed independently and unaffected, which is exactly why this had never been noticed until something finally tried to consume the JSON programmatically.

**UI:**

- **Fixed a page-crashing bug in the Relationship Graph's list view.** `react-force-graph-2d` 's underlying physics simulation mutates its input data in place once it starts (writing position/velocity fields onto every node, and replacing each link's plain-string source/target ids with direct node-object references) — the graph and list view shared the exact same data object, so switching to "View as list" after the graph had already rendered operated on already-mutated data, crashing the entire page with a React "objects are not valid as a child" error. Fixed by giving the force graph its own isolated copy of the data. Live-verified: repeated graph↔list toggling no longer crashes and renders correct data every time.

**Installer (found by a real installation, not a synthetic test):**

- **The shipped Windows installer was stale.** `release/HORIZON-GRID-Setup-0.2.3.exe` had never actually been recompiled from current source — it was missing `docker-compose.yml` / `docker-compose.prod.yml` entirely, so a real, elevated, from-scratch install failed at `docker compose up` with no other symptom ( `exit code 1` ). Fixed by recompiling from current source with a properly installed Inno Setup toolchain; both platform installers ( `.exe` and `.deb` ) are rebuilt fresh for this release.
- **A leftover automated-verification-harness registry entry silently redirected every install to a fake path**, discovered immediately after fixing the bug above (the same symptom recurred on the very next attempt, with the rebuilt installer). Inno Setup's default `UsePreviousAppDir` behavior means it reuses whatever directory the *previous* install on that machine used, regardless of the installer's own configured default — and an earlier automated test run had left a registry `InstallLocation` pointing at a throwaway verification path. Every subsequent install (real or scripted) silently inherited that same fake directory for its file copy, while the Setup Wizard's own path logic correctly used the real `C:\Program Files\...` — a straight mismatch between the two halves of the installer. Fixed by removing the stale registry key; the fix itself required no code change, but any future local

verification harness must now explicitly clean up its own registry trace afterward rather than leaving a persistent `InstallLocation` a real install could inherit.

- **Real AI-response correctness bug found on the resulting live install**: investigating `8.8.8.8` with Ollama returned `threat_assessment: "malicious"` — a bare one-word label, not a sentence — alongside a correctly-computed `final_verdict: "benign"` in the same response. The existing verdict/risk-agreement validator only checked `final_verdict` against the risk numbers; nothing checked `threat_assessment`'s own content. The Final Assessment panel renders the two in separately-labeled tabs, so an analyst reading either one alone would see the platform's opposite conclusion. Fixed with a third, narrowly-scoped validator that only fires when `threat_assessment` is an exact, bare match for a verdict-shaped word contradicting `final_verdict` — confirmed via a live-reproduced regression test, and confirmed *not* to fire on ordinary narrative prose that happens to contain the word "malicious" mid-sentence.

**Testing:**

- New regression tests for every fix in this release that didn't already have coverage (the retry-logic fix, the login rate limiter, the SSRF guard on both the AI-call and config-save paths, and the `threat_assessment` contradiction bug above). Full backend suite: 380 passed, 39 skipped (up from 365 at the start of this review).
- A 3-hour soak test against the live stack completed cleanly: memory flat throughout (no growth trend), zero spontaneous container restarts.
- **A full functional-testing pass against a genuine clean Windows install** (not the dev stack): real elevated installer run, real `docker compose up`, real admin account creation/login, real IOC investigations across five IOC types, real AI-backend switching (specifically checking that a valid-but-wrong credential never misreports as "no API key provided" — it doesn't), real threat scoring, provider health, executive dashboard, a real `nmap` port scan against `127.0.0.1`, real PDF/CSV export, and persistence across a real backend restart and a simulated AI-backend outage (Ollama pointed at an unreachable port — graceful degradation confirmed, full recovery confirmed once restored). This pass is what found the three bugs above. It is explicitly disclosed as an interim, not final, pass — see `FULL_FUNCTIONAL_TEST_REPORT.md` for the complete list of what was and wasn't covered, including that the Setup Wizard's own GUI was driven via its underlying functions directly rather than clicked through, and that a live Windows reboot test, uninstall/reinstall test, and Linux install test were deliberately deferred rather than performed unattended overnight.

**Known limitations (disclosed, not fixed in this release):** no automated host-disk-space alerting; no retention/cleanup job for ever-growing investigation tables; no off-host/off-site backup copy option (backups are local-disk-only); a narrow DNS-rebinding TOCTOU window on the SSRF check (validates a resolution snapshot, the real call re-resolves independently afterward); Neo4j and OpenSearch remain fully provisioned with zero actual application traffic (confirmed via exhaustive search — a real resource cost with no current benefit, flagged as an open product question). See the Mission-Critical Operations Manual for the complete list.

## v0.2.2 — Independent re-verification: 7 real bugs found and fixed

An independent, adversarial re-verification of the v0.2.1 port-scanning cancellation fix (nine parallel reviewers, each reading the real code fresh rather than trusting the prior report) confirmed the cancellation fix itself is correct — the original before/after claim was re-proven directly from git history — but found four new, real defects in the scanner and three unrelated ones surfaced incidentally during full-application regression. All seven are fixed, tested, and live-verified below; none were assumed fixed just because a test suite was green.

**Port scanner (in scope):**

- **IPv6 scans silently never probed the target.** `nmap` requires an explicit `-6` flag for any IPv6 literal target; without it, nmap logged a warning and exited successfully having scanned 0 hosts — previously reported identically to a genuine clean scan (`completed`, 0 findings, no error), indistinguishable from a real result. Fixed by adding `-6` to the argv whenever the target's IOC type is IPv6. Live-reproduced and re-verified: the same `::1` target now correctly comes back "host up, all common ports closed" — a real answer, not a skipped scan.

- **An unknown or mistyped scan profile id was silently accepted and reported as a successful, clean scan.** Neither the API route nor the service layer validated `profile` against the tool's real profile set before creating a run; the bad-profile error was only ever caught deep inside the tool's own code, which returned it as an ordinary (non-exceptional) result — so the run reached `completed` with `error_message: null` and an empty findings list, exactly the "fake result" failure mode this platform's own design principles rule out. Fixed with an explicit profile check alongside the existing tool-id check, rejecting with a clean `400` before any run row is created.
- **A malformed CIDR value could crash the endpoint with a raw 500.** Reachable only via a pre-existing, unrelated lookup-creation override that doesn't itself validate a value against its declared type — but the crash itself was in the scanner's own scope-validation code. Fixed by catching the underlying `ValueError` and converting it to the same clean `400` every other invalid request in this module already produces.
- **A run's own successful completion could silently overwrite a status another writer had already finalized**, discovered via a genuine race during adversarial concurrency testing (a run was marked `failed` by an unrelated recovery process while still genuinely in flight; when it finished for real moments later, the success path unconditionally overwrote that back to `completed` — but left the stale `failed`-only error message in place, producing a self-contradictory row). Fixed by adding the same "only touch a still-pending/running row" guard the cancellation code already used, to all three of the run orchestrator's terminal-state writes. Real scan findings are still always persisted regardless — only the run's own status field is guarded.
- An empty `tool_ids` list is now rejected at the schema level ( `422` ) instead of silently producing a no-op "completed" run.

**Found incidentally during full-application regression (unrelated to port scanning, fixed in the same pass):**

- **Spamhaus DBL/ZEN provider misread a DNS query-rejection as a positive "malicious" verdict.** Spamhaus refuses DNS lookups from public/shared resolvers (a common condition for any containerized deployment using default DNS) with a specific reserved response code — previously classified identically to a real blocklist hit, so a universally benign domain could come back "malicious" purely because of how the deployment's DNS was configured, with no actual finding behind it. Fixed by recognizing the query-rejection codes on both the domain (DBL) and IP (ZEN) lookup paths and reporting them as `unknown` with a clear explanation, never `malicious`; a real listing found alongside a rejection code still correctly reports as malicious — only an error code alone is no longer treated as a positive result.
- **A confusing error on the losing side of a concurrent last-admin-protection race.** The platform already correctly guarantees the system can never reach zero active administrators, even under real concurrency — that invariant held throughout adversarial testing. But the account that loses such a race got a generic "could not validate credentials" (as if their login had broken) rather than a clear explanation that their own account was the one just deactivated by the other request. Fixed by distinguishing "no such account" from "this account was deactivated" and returning the same clear, existing "Account disabled" message used elsewhere for the latter.
- **An AI-generated assessment could cite specific findings in its prose while leaving the structured supporting-evidence list empty** — the exact violation the AI prompt's own instructions already warned against, uncorrected. Since this field holds free-text paraphrased excerpts (not structured IDs), there is no safe way to auto-fill it without risking a fabricated-looking quote; instead, a substantive claim with no supporting evidence now fails validation and reuses the existing retry mechanism, giving the AI a real second chance to either cite its evidence or write a shorter claim that doesn't need it.

## v0.2.1 — Port scanning: real cancellation added

- A forensic audit of the Security Assessment Toolkit's Nmap port/service scanner (prompted by a report that "port scanning is not working correctly") traced the full pipeline end to end — frontend, API route, validation, scanner engine, subprocess execution, result parsing, and UI display — and found each of those stages already working correctly against real local targets. The actual, confirmed gap was narrower and more specific: **there was no way to cancel a running scan.** Once started, a scan (or a whole batch of them) could only be waited out; a wrong profile, a slow/large target, or simply changing your mind had no recourse short of restarting the backend.
- Added a real `POST /api/v1/security-assessment/runs/{run_id}/cancel` endpoint and a "Cancel Scan" button in the Security Assessment panel, visible on any run that's still pending or running. Cancelling genuinely stops the work, not just the displayed

status: the real underlying `nmap` OS process is killed, not left orphaned. Verified live against an actual running scan: cancellation completed in ~1.6 seconds versus the ~12 seconds the same scan would have taken to finish naturally, with no leftover process.

- A cancelled run correctly survives a backend restart: a run left `pending` / `running` by a process that no longer has a live task for it (e.g. after a restart) still resolves to `cancelled` on request, rather than getting stuck permanently with no way to clear it.
- A real, confirmed defect was found and fixed while building this: `asyncio.CancelledError` does not inherit from `Exception` in Python 3.8+, so the scan orchestrator's existing generic error handler silently never caught a cancellation — without a dedicated fix, a cancelled run's database row would have simply stayed `running` forever. A dedicated handler was added, both in the run orchestrator (updates status, writes an audit event) and in the Nmap tool itself (kills the subprocess).
- A real database-migration bug was caught by a genuinely failing test before release, not assumed correct: the new `CANCELLED` status value was initially added to the database enum in lowercase, not matching the existing uppercase convention already used by every other status in that column — corrected and re-verified against a real Postgres instance.
- Cancellation follows the same team-shared-resource permission model as starting a scan ( `security_assessment:create` — any admin or analyst, not only the run's original requester); a VIEWER-role account is correctly blocked with `403` .
- 5 new integration tests cover in-flight cancellation (using a deliberately slow scan so the cancellation genuinely lands mid-flight, not after the scan has already finished), the backend-restart-orphan case, rejecting a cancel on an already-finished run, an unknown run id, and RBAC enforcement.
- A follow-up adversarial pass (racing two concurrent cancel requests for the same run) found a real, low-severity issue: both requests could each write their own `run_cancelled` audit-log entry for what was really one cancellation. Fixed by making the run's own background task the single source of truth for that audit record; a request that arrives after the run is already resolving no longer writes a redundant one. Confirmed live, before and after, by directly querying the audit table during a real concurrent-cancel race.

## v0.2.0 — AI provider ecosystem expansion

- Added five new AI backends, bringing the total from six to eleven: **Kimi** (Moonshot AI), **DeepSeek**, **xAI (Grok)**, **Mistral AI**, and **OpenRouter** (a meta-router giving access to hundreds of underlying models from many providers through one API). Each follows the same forced-tool-calling pattern as the existing OpenAI-compatible backends (Groq, OpenAI), with a real live model-discovery endpoint and a real live connection test — no static-list-only shortcuts.
- Every new backend's real API base URL, auth format, and default model were verified against each provider's live current documentation before implementation, not assumed from prior knowledge — this caught several real, non-obvious details worth calling out: DeepSeek's chat-completions path has no `/v1` segment (unlike every other backend); several of Kimi's newer "thinking"-mode models reject a forced tool call outright, so the default model deliberately avoids them; xAI's error response body is a flat `{"code","error"}` shape rather than the nested shape most other backends use; OpenRouter's live model list is filtered to only models that actually declare `tool_choice` support, since not every model it routes to supports forced tool calling.
- Deliberately did not add every AI provider evaluated. Fireworks AI and Cerebras were researched and found to require real architectural deviations from the existing pattern (Fireworks' model-discovery endpoint needs an account ID, not just an API key; Cerebras' public catalog currently lists only two models and has no documented error-body schema) — both are reasonable future candidates, not a blind exclusion.
- Added 31 new unit tests (connection-test coverage for all 5 new backends: no-key, success, auth failure, model-not-found, rate-limit, and network-timeout paths), plus a Kimi-specific test for the thinking-mode/forced-tool-call conflict. Full backend suite: 313 passed.
- A real, previously-undiscovered Windows installer packaging bug was found and fixed while rebuilding the installer for this release: `installer.iss` never excluded the local `backend/.venv` / `.venv_test` development virtualenvs from the bundled files, so every prior installer build silently included hundreds of megabytes of irrelevant local Python environment files that the actual running application never uses (the real app always builds fresh inside a `python:3.12-slim` container from `requirements.txt` ). Fixing the exclusion dropped the installer from ~69 MB to ~8 MB with zero functional change — this gap was already flagged as a known issue in `linux/build-deb.sh` 's own comments, but never actually fixed until now.

## Rebrand

- Full visible rebrand from "IOC Intelligence Platform" to **HORIZON GRID** (tagline: "Every Signal. One Operational Picture.") across the frontend, backend API title, Windows installer, Start Menu shortcuts, and all documentation.
- Internal identifiers deliberately left unchanged for upgrade safety: the on-disk ProgramData data folder name, the Postgres database name (`ioc_intel`), Python package names, and the Kubernetes namespace. Only user-visible strings were renamed.

## New IOC providers

- Added **urlscan.io** (sandbox category, submit-then-poll scan model) as a new IOC provider.
- Added **Google Safe Browsing** (threat_intel category) as a new IOC provider.
- Both providers were verified, via a real live test with deliberately invalid credentials, to never report a provider failure as a false "safe"/"clean" result — a failed or unreachable call is always surfaced as an error or unknown status.
- A real gap was found and fixed after initial release: both new providers had no live "Test Connection" check wired up (`'<provider>' has no live connection test`) even though the underlying investigation-time provider logic worked correctly. Both now have a real, working connection test.

## Export security fixes

- Fixed a CSV formula-injection vulnerability in exported investigation data (a leading `=` / `+` / `-` / `@` in any exported field is now neutralized).
- Fixed a PDF markup-injection/crash vulnerability in exported investigation reports (all interpolated text is now escaped before reaching the PDF renderer).
- Fixed an export permission-gate bug: exporting an investigation now correctly requires the `lookup:export` permission (previously it was gated on the broader `lookup:read`, which VIEWER-role users also hold — VIEWER cannot export).

## Deterministic threat-scoring engine

- Replaced the previous 100%-AI-generated risk score with a deterministic, versioned, auditable scoring engine (`app/scoring/engine.py`, `SCORING_ENGINE_VERSION` "1.0") — see the dedicated Threat Scoring document for the full formula.
- The AI is now given the deterministic score as a fixed input and can only narrate it; the platform mechanically overwrites the AI's own output with the real score and re-validates the full assessment against it before saving, so an AI model cannot override the number even if it tries to.
- A real audit-trail gap was found and fixed: the engine always computed a full factor breakdown and version stamp, but it was being discarded before the assessment was saved. Both are now persisted on every assessment.
- A real scoring-manipulation vulnerability was found via adversarial testing and fixed: a single free, unprivileged account on a community-sourced provider (AlienVault OTX, ThreatFox, or MalwareBazaar) could previously flood the correlation component with several fabricated, uncorroborated relationship claims and push any indicator's score into the "high" severity band. The correlation component now applies the same cross-provider corroboration discount the provider-vote component already had.

## AI-generation outcome tracking

- Every AI-generated assessment is now tagged with one of three real outcomes: `success`, `skipped_no_evidence` (a correct decision not to call the AI — no provider data existed to analyze, not a failure), or `failed` (a genuine generation failure, with a

real, deterministic-score-based fallback narrative, never a fabricated result). This distinction did not exist before and makes an honest "AI success rate" metric possible.

## Executive Dashboard and Provider Health

- Added a new Executive Dashboard ( `/dashboard` ) with 7 real KPI tiles (active investigations, critical/high-risk IOC count, open/critical case counts, average threat score, provider health percentage, AI success rate) and an AI-generated (or real, number-accurate template-fallback) executive summary, with the fallback explicitly disclosed via a source badge.
- Replaced the old, dead, stub-data `GET /providers/health` endpoint with a real, database-backed implementation reporting per-provider status (healthy/degraded/down/unknown), success rate, average latency, and consecutive-failure streaks across 1-hour/24-hour/7-day/30-day windows, plus a new dedicated Provider Health page.
- A real bug was found and fixed: a provider correctly reporting "nothing found" for an indicator (the normal, healthy outcome for most real-world lookups) was being miscounted as a failure, which could make a perfectly healthy provider appear "degraded" or "down." Confirmed live: one real provider moved from an incorrectly-reported 64% ("Degraded") to a correctly-reported 100% ("Healthy") once fixed.
- A real regression was found and fixed same-day: the new, richer Provider Health response initially dropped a field ( `supported_types` ) an existing page depended on, which would have crashed the live investigation-launch page for every user. Fixed and covered by a regression test before it reached general use.
- A new `dashboard:read` permission was added, deliberately granted to all three roles (ADMIN, ANALYST, VIEWER) for broad, read-only operational visibility.

## Navigation

- Reorganized the global navigation into named groups: COMMAND (Dashboard), INTELLIGENCE (Basket), ANALYSIS (Cases), OPERATIONS (Provider Health), ADMINISTRATION (Providers, Admin — admin-only). No existing route was changed or removed.

## Performance

- Found and fixed a real, complete-failure concurrency bug: 25 concurrent requests to the Provider Health endpoint previously failed 100% of the time (timeout). Root-caused to two issues — roughly 300 sequential database round-trips per single request (consolidated into about 19 via SQL conditional aggregation), and a database connection-pool size that didn't account for every authenticated request holding two connections simultaneously. After both fixes, the same 25-concurrent-request test completes in about 1.6 seconds with 100% success.
- Connection-pool sizing is now tuned per process role: the backend process (the only one serving concurrent dashboard traffic) gets a larger pool; the background worker processes keep a conservative default, since Postgres's own connection ceiling has to be shared across all of them.

## Windows installer

- Fixed a real, reported installer failure: a fresh install could generate a brand-new random database password while an old, incompatible database from an abandoned earlier install attempt survived on the machine, causing the backend to crash-loop on a database authentication error immediately after installation. The installer now detects and clears an orphaned database volume before a genuinely fresh install, so a new password is never paired with an old, incompatible database. An upgrade/reconfigure of a real existing install is unaffected and continues to correctly reuse its real existing password.

## UI

- Added a small persistent attribution/contact bar at the top of every page.

# Linux Release

- Added a first-class Linux release: `horizon-grid_0.2.0_amd64.deb` (~6.1 MB), version 0.2.0 matching Windows exactly, tested on Ubuntu 24.04 LTS, Ubuntu 22.04 LTS, and Debian 12 (bookworm).
- Added a terminal-based CLI setup wizard ( `horizon-grid configure` ) as the Linux analog of the Windows WinForms wizard, covering Administrator Account, AI Configuration, Threat Intelligence Providers, and Network Ports, then writing `/etc/horizon-grid/.env` and bringing up the platform via Docker Compose.
- Added a `horizon-grid` CLI with `configure` , `start` , `stop` , `restart` , `status` , `open` , `backup` , `diagnostics` , `check` , `uninstall` , and `version` commands.
- Added a systemd unit ( `/lib/systemd/system/horizon-grid.service` ), registered on install but never auto-started, and a desktop menu launcher ( `horizon-grid.desktop` , `Exec=horizon-grid open` ).
- Added FHS-standard installed file layout: `/opt/horizon-grid/app` (application), `/etc/horizon-grid/.env` (root:root, chmod 600), `/var/lib/horizon-grid/backups` , `/var/log/horizon-grid/setup.log` .
- Added label-based (not hardcoded-name) cleanup logic for `apt remove` (preserves config/data/volumes) and `apt purge` (removes everything, including all Docker volumes for the Compose project).
- Fixed: excluded two host-only Python virtualenvs ( `.venv` , `.venv_test` , ~21,700 files) from being swept into the Linux build by the packaging script.
- Fixed: `apt purge` / `apt remove` leaving behind an untracked runtime-written `/opt/horizon-grid/app/.env` copy that blocked full directory removal; now cleaned up in the package's `postrm` script.
- Fixed: `horizon-grid backup` using a hardcoded Postgres container name, which silently skipped backups under a non-default Compose project name; now resolves the container via `docker compose ps -q postgres` .